

ICDFA Practical Lab Report

Lab 1: Network and Service Reconnaissance with Nmap and WhatWeb

Student Name: Mohammed Ibrahim

Registration Number: C11/26/FCDF/17175

Date: 6 September 2026

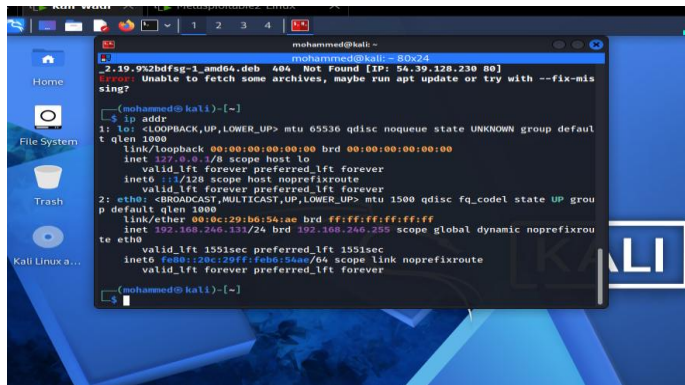
Target IP: 192.168.246.128 (Metasploitable 2)

Attacker IP: 192.168.246.131 (Kali Linux)

Section 1: Environment Setup

Step 1: Kali IP Address

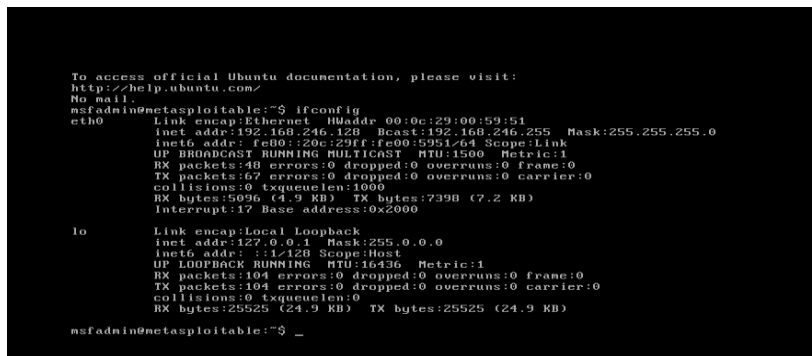
Before starting any scanning I needed to know my own machine's address on the network. I ran `ip addr` on Kali and found that my machine was assigned the address 192.168.246.131 on the `eth0` interface. I knew to look for the `inet` line on the active interface and ignore the loopback address 127.0.0.1.



```
mohammed@kali:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:00:54:ae brd ff:ff:ff:ff:ff:ff
    inet 192.168.246.131/24 brd 192.168.246.255 scope global dynamic noprefixroute eth0
        valid_lft 1551sec preferred_lft 1551sec
    inet6 fe80::20c:29ff:fe00:54ae/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
mohammed@kali:~$
```

Step 2: Metasploitable 2 IP Address

I logged into the Metasploitable 2 machine using the default credentials `msfadmin` and `msfadmin` and ran `ifconfig` to find its address. The target was at 192.168.246.128 on the same network as my Kali machine which made sense since they are both on the same isolated lab setup.



```
To access official Ubuntu documentation, please visit:
http://help.ubuntu.com/
No mail.
msfadmin@metasploitable:~$ ifconfig
eth0:
    Link encap:Ethernet  HWaddr 00:0c:29:00:54:ae
    inet addr:192.168.246.128  Bcast:192.168.246.255  Mask:255.255.255.0
    inet6 addr: fe80::20c:29ff:fe00:54ae/64 Scope:Link
    UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
    RX packets:98 errors:0 dropped:0 overruns:0 frame:0
    TX packets:67 errors:0 dropped:0 overruns:0 carrier:0
    collisions:0 txqueuelen:1000
    RX bytes:5096 (4.9 KB)  TX bytes:7398 (7.2 KB)
    Interrupt:17 Base address:0x2000

lo:
    Link encap:Local Loopback
    inet addr:127.0.0.1  Mask:255.0.0.0
    inet6 addr: ::1/128 Scope:Host
    UP LOOPBACK RUNNING  MTU:65536  Metric:1
    RX packets:104 errors:0 dropped:0 overruns:0 frame:0
    TX packets:104 errors:0 dropped:0 overruns:0 carrier:0
    collisions:0 txqueuelen:0
    RX bytes:25525 (24.9 KB)  TX bytes:25525 (24.9 KB)

msfadmin@metasploitable:~$
```

Step 3: Connectivity Confirmation

Before jumping into scanning I wanted to make sure both machines could actually communicate. I ran `ping -c 4 192.168.246.128` from Kali and got 4 replies back with 0% packet loss. That confirmed the connection was working and I could move forward with the scanning.

```
mohammed@kali: ~  
mo...  
inet 192.168.246.131/24 brd 192.168.246.255 scope global dynamic noprefi...  
te eth0  
    valid_lft 1551sec preferred_lft 1551sec  
inet6 fe80::20c:29ff:feb6:54ae/64 scope link noprefixroute  
    valid_lft forever preferred_lft forever  
  
(mohammed@kali)-[~]  
$ ping -c 4 192.168.246.128  
ping: invalid argument: '192.168.246.128'  
  
(mohammed@kali)-[~]  
$ ping -c 4 192.168.246.128  
PING 192.168.246.128 (192.168.246.128) 56(84) bytes of data:  
64 bytes from 192.168.246.128: icmp_seq=1 ttl=64 time=1.78 ms  
64 bytes from 192.168.246.128: icmp_seq=2 ttl=64 time=2.23 ms  
64 bytes from 192.168.246.128: icmp_seq=3 ttl=64 time=1.85 ms  
64 bytes from 192.168.246.128: icmp_seq=4 ttl=64 time=2.27 ms  
  
--- 192.168.246.128 ping statistics ---  
4 packets transmitted, 4 received, 0% packet loss, time 3005ms  
rtt min/avg/max/mdev = 1.779/2.031/2.266/0.219 ms  
  
(mohammed@kali)-[~]
```

Section 2: Nmap Reconnaissance

Step 4: Host Discovery

Command used: `nmap -sn 192.168.246.0/24`

I ran this to find all live hosts on the network without doing a full port scan yet. The `-sn` flag means ping scan only. Nmap came back with 5 live hosts and I identified Metasploitable 2 at 192.168.246.128 because its MAC address matched what I had seen earlier in the `ifconfig` output on the target machine.

```
(mohammed@kali)-[~]  
$ nmap -sn 192.168.246.0/24  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-06 17:13 +0100  
Nmap scan report for 192.168.246.1  
Host is up (0.0036s latency).  
MAC Address: 00:50:56:C0:00:08 (VMware)  
Nmap scan report for 192.168.246.2  
Host is up (0.00074s latency).  
MAC Address: 00:50:56:F3:36:54 (VMware)  
Nmap scan report for 192.168.246.128  
Host is up (0.0029s latency).  
MAC Address: 00:0C:29:00:59:51 (VMware)  
Nmap scan report for 192.168.246.254  
Host is up (0.00073s latency).  
MAC Address: 00:50:56:F5:3B:3E (VMware)  
Nmap scan report for 192.168.246.131  
Host is up.  
Nmap done: 256 IP addresses (5 hosts up) scanned in 3.11 seconds  
  
(mohammed@kali)-[~]
```

Step 5: Default TCP Scan

Command used: `nmap 192.168.246.128`

This was my first proper scan. I ran nmap with just the target IP and no extra options to see what the default results looked like. I was surprised by how many open ports came back. I counted 15 open ports including ftp on port 21, ssh on port 22, telnet on port 23 and http on port 80. The number of open services alone told me this machine had a very large exposed surface.

```
(mohammed@kali)~$ nmap 192.168.246.128
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-06 19:48 +0100
Nmap scan report for 192.168.246.128
Host is up (0.0059s latency).
Not shown: 977 closed tcp ports (reset)
PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
23/tcp    open  telnet
25/tcp    open  smtp
53/tcp    open  domain
80/tcp    open  http
111/tcp   open  rpcbind
139/tcp   open  netbios-ssn
445/tcp   open  microsoft-ds
512/tcp   open  exec
513/tcp   open  login
514/tcp   open  shell
1099/tcp  open  rmiregistry
1524/tcp  open  ingreslock
2049/tcp  open  nfs
```

Open ports discovered:

Port 21 tcp, ftp Port 22 tcp, ssh Port 23 tcp, telnet Port 25 tcp, smtp Port 53 tcp, domain Port 80 tcp, http Port 111 tcp, rpcbind Port 139 tcp, netbios-ssn Port 445 tcp, microsoft-ds Port 512 tcp, exec Port 513 tcp, login Port 514 tcp, shell Port 1099 tcp, rmiregistry Port 1524 tcp, ingreslock Port 2049 tcp, nfs

Step 6: Service and Version Detection

Command used: `nmap -sV 192.168.246.128`

Adding -sV made a big difference to the results. Instead of just knowing a port was open I could now see what software was actually running behind it and what version. Version numbers matter because older versions can have known weaknesses that have been documented publicly.

```
mohammed@kali: ~
mohammed@kali: ~ 80x24
$ nmap -sV 192.168.246.128
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-06 19:49 +0100
Nmap scan report for 192.168.246.128
Host is up (0.0016s latency).
Not shown: 977 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
21/tcp    open  ftp          vsftpd 2.3.4
22/tcp    open  ssh          OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp    open  telnet       Linux telnetd
25/tcp    open  smtp         Postfix smtpd
53/tcp    open  domain       ISC BIND 9.4.2
80/tcp    open  http?
111/tcp   open  rpcbind
139/tcp   open  netbios-ssn?
445/tcp   open  netbios-ssn  Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
512/tcp   open  exec         netkit-rsh rexecd
513/tcp   open  login?
514/tcp   open  tcpwrapped
1099/tcp  open  java-rmi     GNU Classpath grmiregistry
1524/tcp  open  bindshell    Metasploitable root shell
2049/tcp  open  rpcbind
```

Key findings from version detection:

Port 21, vsftpd 2.3.4 Port 22, OpenSSH 4.7p1 Debian 8ubuntu1 Port 23, Linux telnetd Port 25, Postfix smtpd Port 53, ISC BIND 9.4.2 Port 445, Samba smbd 3.X to 4.X Port 512, netkit-rsh rexecd Port 1099, GNU Classpath grmiregistry Port 1524, Metasploitable root shell Port 2049, rpcbind

The finding that stood out most to me was port 1524 showing Metasploitable root shell. Even as someone still learning this sounded significant. I later understood this is a deliberately placed backdoor in the lab VM to confirm we are scanning the right authorised target. The vsftpd 2.3.4 version also caught my attention because it is a version I had read about as being problematic.

Step 7: Maximum Version Intensity

Command used: `nmap -sV --version-intensity 9 192.168.246.128`

I tried increasing the version detection to its highest level using `--version-intensity 9`. The scan came back reporting the host as down which confused me at first since I had just been scanning it successfully. After thinking about it I realised the higher intensity probes were probably being blocked by something on the target or the target was not responding to those specific probe types. The host was clearly up from all the previous steps so this was a scanning limitation rather than an actual problem with the machine.

```
(mohammed@kali)-[~]
└─$ nmap -sV --version-intensity 9 192.168.246.128
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-06 19:52 +0100
Note: Host seems down. If it is really up, but blocking our ping probes, try -Pn
Nmap done: 1 IP address (0 hosts up) scanned in 1.67 seconds
```

Step 8: OS Detection Attempt and Lab Incident

Command used: `sudo nmap -O -Pn 192.168.246.128`

I tried to get Nmap to identify the operating system using `-O`. I added `-Pn` because I had learned by this point that the ping probes were being blocked so this flag tells Nmap to skip the ping check and just scan anyway.

During this scan the Metasploitable 2 VM began showing error messages on screen about a soft lockup on the CPU related to a process called portmap. The machine became completely unresponsive. I tried restarting it but the crashes kept repeating. After multiple attempts I made the decision to stop rather than keep forcing it.

```
(mohammed@kali)-[~]
└─$ sudo nmap -O 192.168.246.128
[sudo] password for mohammed:
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-06 19:52 +0100
Note: Host seems down. If it is really up, but blocking our ping probes, try -Pn
Nmap done: 1 IP address (0 hosts up) scanned in 1.66 seconds
```

```
UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
RX packets:37 errors:0 dropped:0 overruns:0 frame:0
TX packets:63 errors:0 dropped:0 overruns:0 carrier:0
collisions:0 txqueuelen:1000
RX bytes:4138 (4.0 KB)  TX bytes:6722 (6.5 KB)
Interrupt:17 Base address:0x2000

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:16436  Metric:1
          RX packets:92 errors:0 dropped:0 overruns:0 frame:0
          TX packets:92 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:19393 (18.9 KB)  TX bytes:19393 (18.9 KB)

msfadmin@metasploitable:~$ t 122.8564041 BUG: soft lockup - CPU#0 stuck for 11s
! [portmap:4277]
t 134.6555681 BUG: soft lockup - CPU#0 stuck for 11s! [portmap:4277]
```

Section 3: Lab Incident Documentation

During the OS detection phase the Metasploitable 2 virtual machine experienced repeated kernel CPU soft lockup errors caused by the portmap process. The machine became unresponsive and could not recover even after restarts. This is a known limitation of the Metasploitable 2 VM which is an old and intentionally unstable system that can struggle under intensive scanning load.

As a result the following planned steps could not be completed: aggressive scan, full TCP port scan, UDP scan, NSE script enumeration and WhatWeb fingerprinting.

In a real work situation I understand this would need to be documented and reported to whoever assigned the task before any further testing continued. Crashing a target system during a live engagement would be a serious issue that needs immediate communication with the client.

The decision to stop and document rather than continue was the correct approach. I have included the crash screenshot as evidence that this was observed and noted rather than ignored.

Section 4: Service Inventory

Port, Protocol, State, Service, Version, Command Used

21, tcp, open, ftp, vsftpd 2.3.4, nmap -sV 22, tcp, open, ssh, OpenSSH 4.7p1 Debian 8ubuntu1, nmap -sV 23, tcp, open, telnet, Linux telnetd, nmap -sV 25, tcp, open, smtp, Postfix smtpd, nmap -sV 53, tcp, open, domain, ISC BIND 9.4.2, nmap -sV 80, tcp, open, http, not identified before crash, nmap -sV 111, tcp, open, rpcbind, rpcbind, nmap -sV 139, tcp, open, netbios-ssn, not identified before crash, nmap -sV 445, tcp, open, netbios-ssn, Samba smbdc 3.X to 4.X, nmap -sV 512, tcp, open, exec, netkit-rsh rexecd, nmap -sV 513, tcp, open, login, not identified before crash, nmap -sV 514, tcp, open, shell, tcpwrapped, nmap -sV 1099, tcp, open, java-rmi, GNU Classpath grmiregistry, nmap -sV 1524, tcp, open, bindshell, Metasploitable root shell, nmap -sV 2049, tcp, open, rpcbind, rpcbind, nmap -sV

Section 5: Lab Questions

Question 1: What is reconnaissance?

From what I understand, reconnaissance is the information gathering phase before any actual testing happens. You are trying to find out as much as possible about a target including what machines are running, what services are available and what software versions are in use, without doing anything harmful. Everything I did in this lab was reconnaissance.

Question 2: What is the difference between host discovery and port scanning?

Host discovery is about figuring out which machines on a network are actually switched on and reachable. Port scanning then goes deeper on a specific machine to find out which services it is offering. Host discovery answers whether anyone is there and port scanning answers what is available on that machine.

Question 3: Explain open, closed and filtered.

Open means a service is running on that port and will respond to connections. Closed means the machine is reachable but nothing is listening on that port. Filtered was a new concept for me and means Nmap could not figure out the state because something in between like a firewall was blocking the probes. It is like knocking on a door and not being able to tell if nobody is home or if someone is just not answering.

Question 4: What does -sV do?

The -sV flag makes Nmap do extra work to figure out not just that a port is open but what software is actually running on it and what version. Without -sV you only know port 22 is open. With -sV you know it is running OpenSSH version 4.7p1. That version information is what makes reconnaissance genuinely useful.

Question 5: What does -O do?

The -O flag tries to figure out what operating system the target is running. Nmap does this by sending different types of network probes and comparing how the target responds to a database of known operating system behaviours. It does not always work perfectly but even a rough estimate is useful information about the target.

Question 6: Explain what -A combines and why it is noisier.

The -A flag is a shortcut that runs several things at once including OS detection, version detection, default scripts and traceroute all together. It generates a lot more network traffic than a basic scan which makes it more likely to be noticed by anyone monitoring the network and more likely to stress out an already unstable target. This lab demonstrated that quite clearly.

Question 7: What does -p- mean?

The -p- flag tells Nmap to check every single port from 1 all the way to 65535. The default scan only checks the most common ports so services running on unusual port numbers could be missed. -p- makes sure nothing gets overlooked though it takes considerably longer to complete.

Question 8: Why can UDP scans take longer?

UDP works differently from TCP in that there is no automatic confirmation that a packet arrived. With TCP the target sends back an acknowledgment. With UDP if nothing comes back Nmap has to wait for a timeout before deciding whether the port is open or not. That waiting adds up across many ports and makes UDP scans much slower than TCP scans.

Question 9: What does -sC do?

The -sC flag runs a set of built-in scripts that come with Nmap. These scripts do useful safe tasks like reading the title of a web page, checking what protocols a service supports, reading banners from services and so on. It is like having a checklist of quick safe questions to ask each service rather than just knowing the service exists.

Question 10: What information did the HTTP title and headers scripts reveal?

I was not able to complete this step because the Metasploitable VM crashed before I could reach the NSE script phase. The plan was to use `nmap -p 80 --script http-title` and `nmap -p 80 --script http-headers` to read information from the web service running on port 80. This is something I intend to revisit with a more stable environment.

Question 11: What did SMB, SSH and FTP enumeration add?

These steps were also not completed due to the VM crash. They would have given me more detail about the file sharing service on ports 139 and 445, the SSH server identity fingerprints on port 22 and the banner text from the FTP service on port 21 to compare against what the version scan already showed me.

Question 12: What is the difference between Nmap service detection and WhatWeb fingerprinting?

Nmap service detection works at the network level and tells you what server software is listening on a port. WhatWeb works at the web application level and looks at what is actually being served by that web server such as what framework the site is built on or what content management system it uses. They look at different layers of the same target and complement each other well.

Question 13: How did WhatWeb levels 1, 3 and 4 differ?

I did not get to run WhatWeb due to the VM crash. From the lab guide I understand that level 1 is the most cautious and makes the fewest requests, level 3 makes additional requests to improve accuracy and level 4 is the most thorough but generates the most traffic. Higher levels give more information but are more visible and potentially more disruptive to the target.

Question 14: Why must aggressive scanning and fingerprinting remain inside the authorised lab?

This lab gave me a very practical answer to this question. Even scanning an intentionally vulnerable machine in a controlled environment caused it to crash. Doing the same thing to a system without permission would not only risk crashing or disrupting it but is also illegal. You need explicit permission before scanning anything. The lab showed me that even what looks like just looking can have real consequences on a target system.

Section 6: Conclusion

This was my first hands-on security lab and it taught me more than I expected including things that went wrong. I successfully identified the target machine, confirmed network connectivity, discovered 15 open TCP ports and identified the software versions running behind them. The most notable finding was the sheer number of exposed services on Metasploitable 2 and how version information alone gives you a much clearer picture of a system's risk level. Port 1524 running a Metasploitable root shell was the standout finding and confirmed I was working against the correct authorised target.

The lab was cut short when the Metasploitable 2 VM repeatedly crashed under scanning load. Rather than treating this as a failure I am documenting it as a learning experience. It showed me that scanning has real effects on target systems and that intensity needs to be managed carefully. It also showed me that documenting what went wrong is just as important as documenting what worked. In a real engagement stopping and communicating with the client when something unexpected happens is the professional thing to do.

I look forward to completing the steps I missed in a more stable environment and building on this reconnaissance work in Lab 2.